

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук, доцент

должность, уч. степень, звание

подпись, дата

О. И. Красильникова

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7
СОЗДАНИЕ СЦЕНАРИЕВ НА JAVASCRIPT

по курсу: Web-технологии

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

С использованием языка JavaScript научиться создавать различные сценарии, выполняемые в web-страницах.

2 Задание

Выполнить общую часть лабораторной работы: подключить внешний скрипт с приветственным сообщением alert и реализовать одну и ту же функцию тремя способами (Function Declaration, Function Expression, Arrow Function) с вызовом по клику на соответствующих кнопках.

Выполнить вариант 1 второй части задания:

1) создать веб-страницу с кнопкой «Напоминания», по клику на которой последовательно выводятся модальные окна типа alert с напоминаниями о делах до заданных сроков;

2) создать веб-страницу с контейнером, содержащим два параграфа с текстом и кнопку, по щелчку на которой после первого контейнера на странице появится еще один контейнер, в котором первый абзац будет содержать прежний текст, а во втором параграфе — другой текст (использовать клонирование узлов)

3 Результат работы

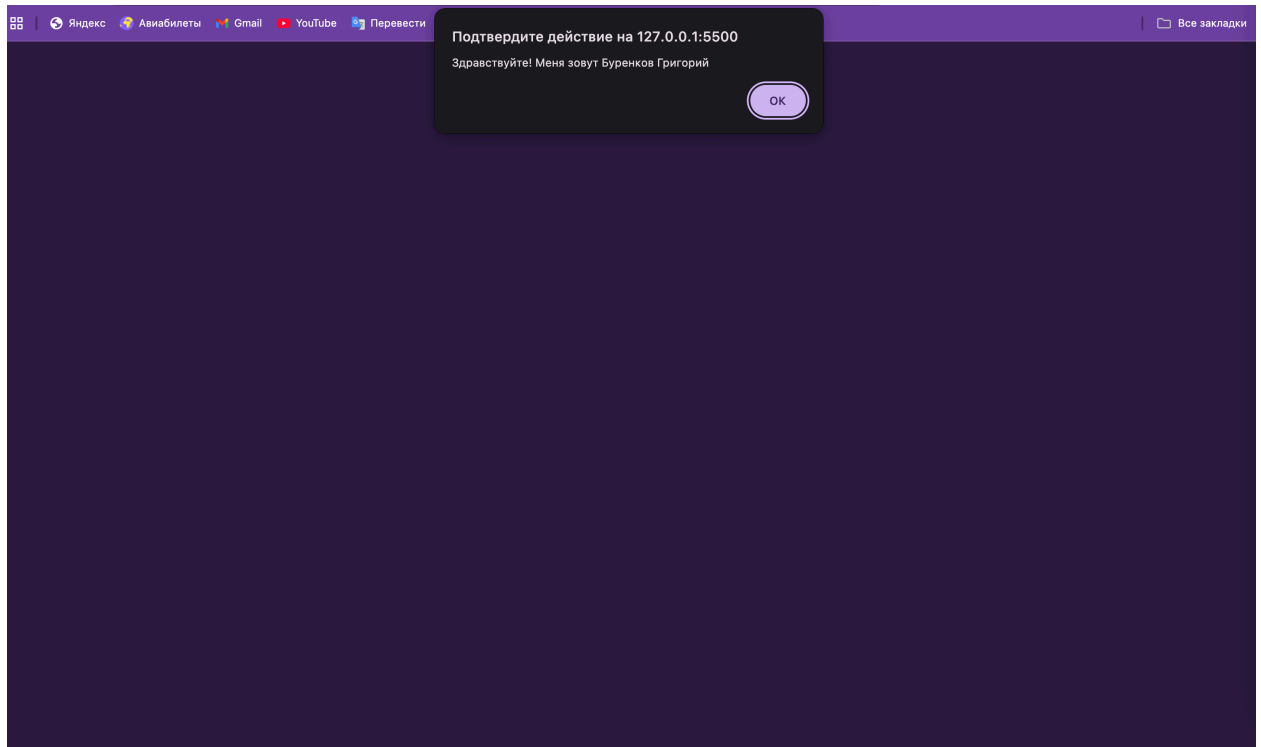


Рисунок 1 – Вход на страницу (алерт)

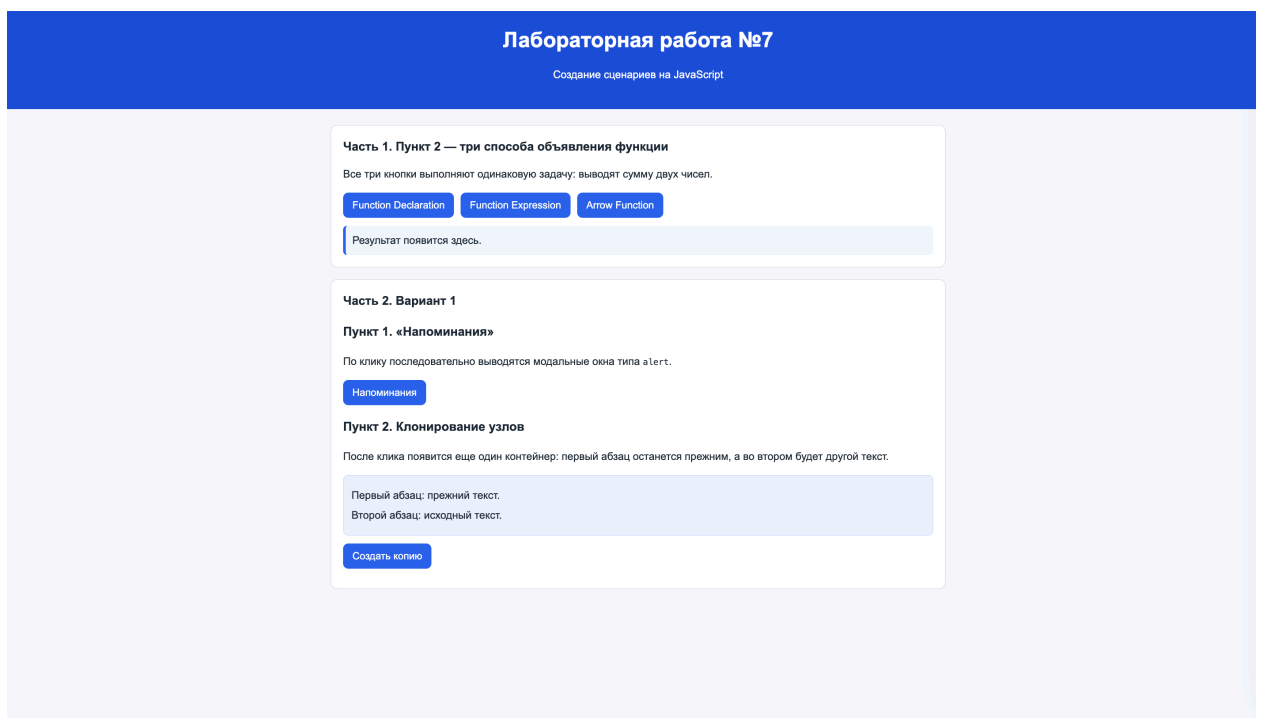


Рисунок 2 – Результат страницы

Часть 1. Пункт 2 — три способа объявления функции

Все три кнопки выполняют одинаковую задачу: выводят сумму двух чисел.

Function Declaration

Function Expression

Arrow Function

Function Declaration: $12 + 8 = 20$

Рисунок 3 – Declaration

Часть 1. Пункт 2 — три способа объявления функции

Все три кнопки выполняют одинаковую задачу: выводят сумму двух чисел.

Function Declaration

Function Expression

Arrow Function

Function Expression: $12 + 8 = 20$

Рисунок 4 – Expression

Часть 1. Пункт 2 — три способа объявления функции

Все три кнопки выполняют одинаковую задачу: выводят сумму двух чисел.

Function Declaration

Function Expression

Arrow Function

Arrow Function: $12 + 8 = 20$

Рисунок 5 – Arrow Function

Пункт 2. Клонирование узлов

После клика появится еще один контейнер: первый абзац останется прежним, а во втором будет другой текст.

Первый абзац: прежний текст.

Второй абзац: исходный текст.

Первый абзац: прежний текст.

Второй абзац: текст во втором контейнере (измененный).

Создать копию

Рисунок 6– Клонирование узлов

4 Вывод

В ходе лабораторной работы были закреплены базовые навыки JavaScript для динамического изменения содержимого страницы и обработки пользовательских событий. Были реализованы три формы объявления функций, что позволило сравнить синтаксис и получить одинаковый результат выполнения одной задачи разными способами. Также освоено назначение обработчиков через `addEventListener`, последовательный вывод уведомлений через `alert` и клонирование DOM-узлов с последующим изменением текста. Поставленная цель лабораторной работы достигнута.

ПРИЛОЖЕНИЕ А

Код файла style.css:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f6fb;
  color: #1f2937;
  line-height: 1.5;
}

header {
  background: #1d4ed8;
  color: #fff;
  padding: 24px 16px;
  text-align: center;
}

header h1 {
  margin: 0 0 8px;
}

main {
  max-width: 960px;
  margin: 24px auto;
  padding: 0 16px 24px;
}

.task-block {
  background: #fff;
  border: 1px solid #d1d5db;
  border-radius: 10px;
  padding: 18px;
  margin-bottom: 18px;
}

.task-block h2 {
  margin-top: 0;
  font-size: 1.15rem;
}

.button-row {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
  margin: 12px 0;
}

button {
  border: none;
  border-radius: 8px;
  padding: 10px 14px;
  cursor: pointer;
  background: #2563eb;
  color: #fff;
  font-size: 0.95rem;
}

button:hover {
  background: #1e40af;
}

.result {
  background: #eff6ff;
```

```

border-left: 4px solid #2563eb;
padding: 10px;
border-radius: 6px;
margin: 0;
}

.clone-container {
background: #eef2ff;
border: 1px solid #c7d2fe;
border-radius: 8px;
padding: 12px;
margin: 12px 0 6px;
}

.clone-p1,
.clone-p2 {
margin: 6px 0;
}
} Код файла index.html:
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ЛР7 – JavaScript сценарии</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Лабораторная работа №7</h1>
    <p>Создание сценариев на JavaScript</p>
  </header>

  <main>
    <section class="task-block">
      <h2>Часть 1. Пункт 2 – три способа объявления функции</h2>
      <p>Все три кнопки выполняют одинаковую задачу: выводят сумму двух
чисел.</p>
      <div class="button-row">
        <button id="declarationBtn">Function Declaration</button>
        <button id="expressionBtn">Function Expression</button>
        <button id="arrowBtn">Arrow Function</button>
      </div>
      <p id="functionResult" class="result">Результат появится здесь.</p>
    </section>

    <section class="task-block">
      <h2>Часть 2. Вариант 1</h2>

      <h3>Пункт 1. «Напоминания»</h3>
      <p>По клику последовательно выводятся модальные окна типа
<code>alert</code>.</p>
      <div class="button-row">
        <button id="remindersBtn" type="button">Напоминания</button>
      </div>

      <h3>Пункт 2. Клонирование узлов</h3>
      <p>После клика появится еще один контейнер: первый абзац останется
прежним, а во втором будет другой текст.</p>

      <div id="cloneContainer1" class="clone-container">
        <p class="clone-p1">Первый абзац: прежний текст.</p>
        <p class="clone-p2">Второй абзац: исходный текст.</p>
      </div>
      <div class="button-row">
        <button id="cloneBtn" type="button">Создать копию</button>
      </div>
    </section>
  </main>

  <script src="script.js"></script>

```



```
</body>
</html>
```

Код файла index.html:

```
alert("Здравствуйте! Меня зовут Буренков Григорий");

// Часть 1. Три способа объявления функции (одна и та же задача)
function sumByDeclaration(a, b) {
    return a + b;
}

const sumByExpression = function (a, b) {
    return a + b;
};

const sumByArrow = (a, b) => a + b;

const functionResult = document.getElementById("functionResult");
const a = 12;
const b = 8;

document.getElementById("declarationBtn").addEventListener("click", () => {
    functionResult.textContent = `Function Declaration: ${a} + ${b} =
    ${sumByDeclaration(a, b)}`;
});

document.getElementById("expressionBtn").addEventListener("click", () => {
    functionResult.textContent = `Function Expression: ${a} + ${b} =
    ${sumByExpression(a, b)}`;
});

document.getElementById("arrowBtn").addEventListener("click", () => {
    functionResult.textContent = `Arrow Function: ${a} + ${b} = ${sumByArrow(a, b)}`;
});

// Часть 2, вариант 1, пункт 1: поочередные модальные окна alert
const remindersBtn = document.getElementById("remindersBtn");
const reminders = [
    "Напоминание: до 1-го числа продлить проездную карту.",
    "Напоминание: до 10-го числа оплатить жилищно-коммунальные услуги.",
    "Напоминание: до 25-го числа ввести показания счетчиков.",
    "Напоминание: записаться к врачу и проверить план обследований."
];

remindersBtn.addEventListener("click", () => {
    // Важно: alert блокирует выполнение, поэтому можно вывести все напоминания
    // в одном обработчике клика "поочередно".
    for (const reminder of reminders) {
        alert(reminder);
    }

    remindersBtn.disabled = true;
    remindersBtn.textContent = "Напоминания выполнены";
});

// Часть 2, вариант 1, пункт 2: клонирование узлов (создание второго контейнера)
const cloneContainer1 = document.getElementById("cloneContainer1");
const cloneBtn = document.getElementById("cloneBtn");

cloneBtn.addEventListener("click", () => {
    // Клонировем контейнер вместе с содержимым и меняем только второй абзац.
    const newContainer = cloneContainer1.cloneNode(true);
    newContainer.id = `cloneContainer_${Date.now()}`;

    const p2 = newContainer.querySelector(".clone-p2");
    if (p2) {
        p2.textContent = "Второй абзац: текст во втором контейнере (измененный).";
    }

    cloneContainer1.insertAdjacentElement("afterend", newContainer);
});
```

});